

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	K.MAHESH BABU	Reg. No.:	192525122
Section / Batch:	CSA1214/2025	Date:	25/08/26

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Design an I/O subsystem for a real-time robotic arm controller with the following constraints:
 - Position sensors generate data every 500 μ s.
 - Control commands must be processed within 5 μ s.
 - Use priority-based interrupt handling for critical sensors.
 - Non-critical diagnostic data must be transferred using DMA.
2. Construct a communication architecture for an autonomous vehicle with the following constraints:
 - Multiple cameras generate a combined data rate of 2 GB/s.
 - Safety-critical sensors require deterministic response time.
 - Use PCIe for high-bandwidth devices.
 - Ensure that low-priority devices do not affect safety-critical communication.
3. Design an I/O mechanism for an industrial monitoring system with the following constraints:
 - 100 sensors continuously generate measurement data.
 - The system must support real-time fault detection.
 - Use interrupt coalescing to reduce processor overhead.
 - Critical fault signals must receive immediate interrupt service.
4. Construct a high-speed network interface subsystem with the following constraints:
 - Network traffic can reach 10 Gb/s.
 - The CPU must be protected from excessive packet-processing overhead.
 - Use DMA-based packet transfer between the network interface and memory.
 - Generate interrupts only after processing a configurable batch of packets.
5. Design an I/O architecture for a smart healthcare monitoring system with the following constraints:
 - Patient-monitoring devices generate data at different sampling rates.
 - Emergency signals must be handled with highest priority.
 - Use nested interrupt handling for urgent events.
 - Store continuous monitoring data using DMA transfers without interrupting critical processing.

Code of Conduct
/ K.MAHESHBABU(192525122)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.MAHESHBABU(192525122)

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 - Exemplary (5 Marks)	Level 3 - Proficient (4 Marks)	Level 2 - Developing (2-3 Marks)	Level 1 - Beginning (0-1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

Q1. Design an I/O subsystem for a real-time robotic arm controller

Given Constraints

- Position sensors generate data every 500 μs.
- Control commands must be processed within 5 μs.
- Priority-based interrupt handling must be used for critical sensors.
- Non-critical diagnostic data must be transferred using DMA.

1. Problem Understanding

The robotic arm is a real-time system. It receives continuous position information from sensors and must respond to critical sensor information within a very short time. The most important requirement is the 5 μs control-response deadline. Therefore, the I/O subsystem must:

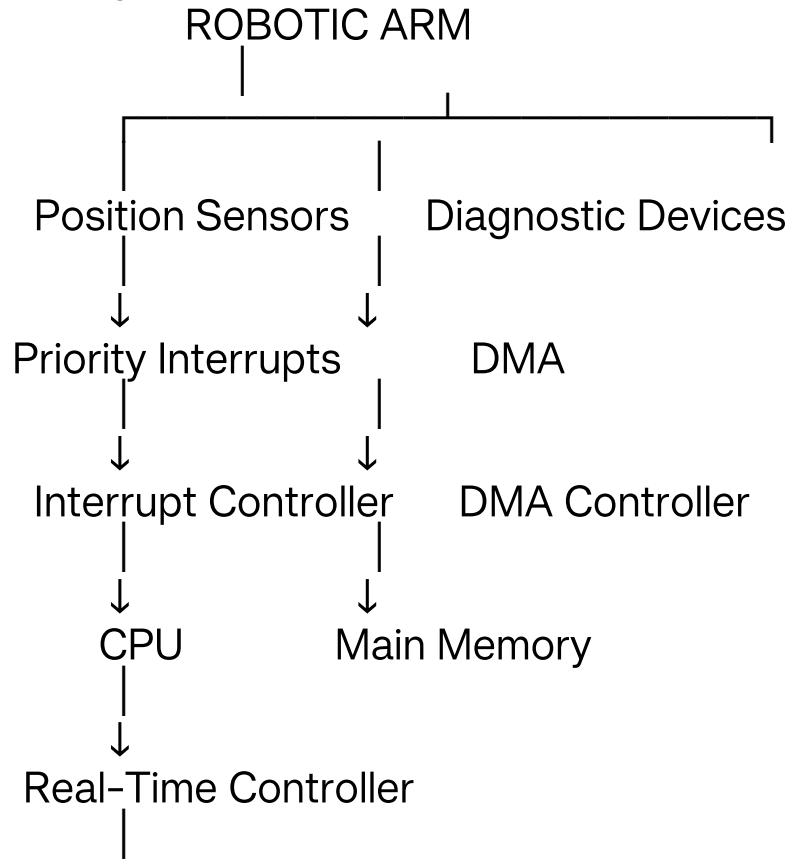
1. Process critical sensor data with minimum latency.
2. Give higher priority to critical sensors.
3. Prevent diagnostic data from consuming excessive CPU time.
4. Use DMA for non-critical data.
5. Maintain continuous sensor monitoring.

2. Constraint Analysis

Constraint	Requirement	Priority
Sensor interval	Every 500 μs	High
Control response	≤ 5 μs	Critical
Critical sensors	Priority interrupts	Critical
Diagnostic data	DMA transfer	Medium
CPU utilization	Must be minimized	High

The control-command deadline is the highest-priority constraint because missing it can affect robotic-arm stability and safety.

3. Proposed I/O Architecture



↓
Motor Actuators

4. Operation

Step 1: Sensor Data Generation

The position sensors generate data every:

Therefore, the sensor frequency is:

So each sensor generates 2000 data events per second.

Step 2: Critical Sensor Interrupt

When a critical sensor generates data:

Sensor



Priority Interrupt



Interrupt Controller



CPU



Control Algorithm



Motor Command

The interrupt mechanism ensures that critical sensor information receives immediate CPU attention.

Step 3: Diagnostic Data

Non-critical diagnostic information does not require immediate CPU processing.

Therefore:

Diagnostic Device



DMA



Main Memory

The DMA controller transfers data directly between the device and memory without requiring the CPU to handle every individual transfer.

5. Priority Scheme

A suitable priority structure is:

Priority 1 → Emergency / Critical Position Sensor

Priority 2 → Other Position Sensors

Priority 3 → Control Status

Priority 4 → Diagnostic Data

If a lower-priority diagnostic operation is occurring and a critical sensor generates an interrupt, the critical interrupt is serviced first.

6. Application of Concepts

Priority-Based Interrupts

Used for critical sensor signals.

Benefit: Reduces response time.

DMA

Used for diagnostic data.

Benefit: Reduces CPU involvement.

Interrupt Controller
Determines which interrupt should be serviced first.
Buffer
A memory buffer can temporarily store diagnostic data transferred by DMA.

7. Justification

The architecture satisfies all constraints:

- Sensor data is continuously monitored.
- Critical sensor information receives priority.
- Control commands can be processed with minimum interrupt delay.
- Diagnostic transfers use DMA.
- CPU resources are protected from unnecessary diagnostic interrupts.

The 500 μs sensor interval provides:
Thus, the system has a 5 μs response requirement within a 500 μs sensor-generation interval, requiring deterministic priority handling.

Conclusion
The proposed architecture using priority-based interrupts for critical sensors and DMA for diagnostic data is suitable for the real-time robotic arm. Critical operations receive immediate CPU attention, while DMA reduces CPU overhead from non-critical transfers. This provides low latency, efficient CPU utilization, and reliable real-time control.

Q2. Construct a communication architecture for an autonomous vehicle

Given Constraints

- Multiple cameras generate a combined data rate of 2 GB/s.
- Safety-critical sensors require deterministic response time.
- PCIe must be used for high-bandwidth devices.
- Low-priority devices must not affect safety-critical communication.

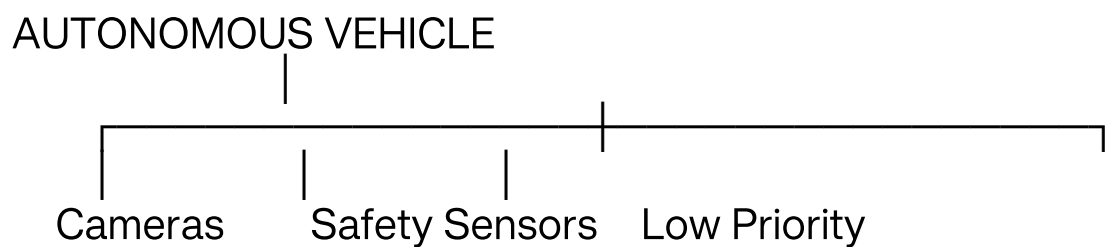
1. Problem Understanding

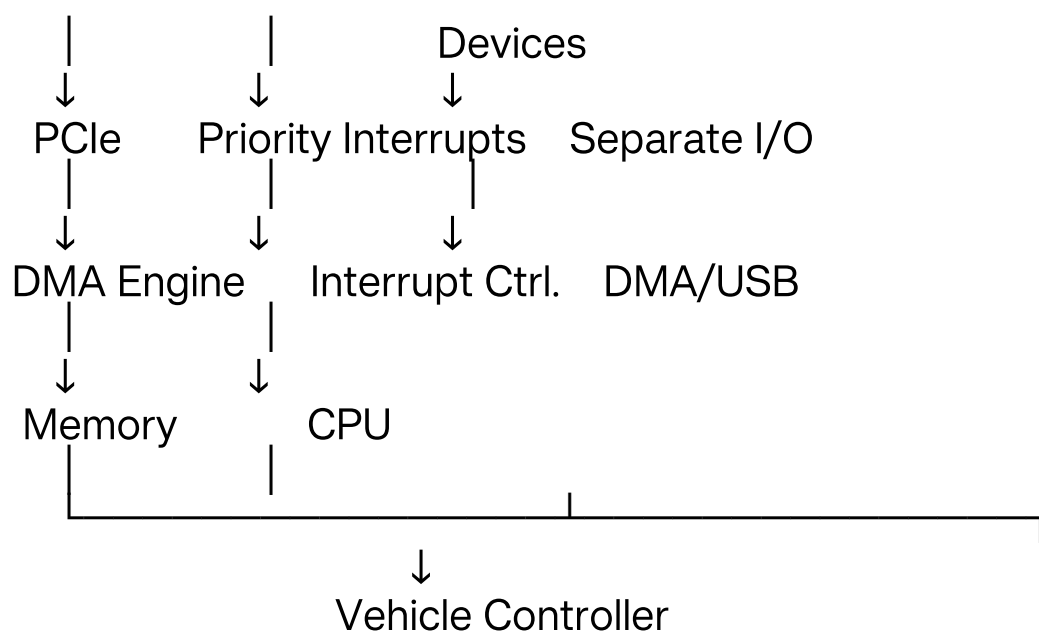
An autonomous vehicle processes data from cameras and safety-critical sensors.
The camera system produces:
This is a high-bandwidth requirement.
At the same time, safety-critical sensors require deterministic and predictable response time.
Therefore, the architecture must separate high-bandwidth camera traffic from safety-critical traffic.

2. Constraint Analysis

Constraint	Required Solution	Priority
Camera data	2 GB/s	High
High-bandwidth devices	PCIe	High
Safety sensors	Deterministic response	Critical
Low-priority devices	Must not interfere	High
CPU overhead	Should be minimized	High

3. Proposed Architecture





4. Camera Data Calculation

Camera data rate:
 Converting to bits:
 Therefore:
 The communication subsystem must therefore provide sufficient bandwidth for approximately 16 Gb/s of camera data.
 PCIe is selected because the problem specifically requires PCIe for high-bandwidth devices.

5. Safety-Critical Communication

Safety sensors should use:
 Safety Sensor
 ↓
 Priority Interrupt
 ↓
 Interrupt Controller
 ↓
 CPU
 ↓
 Safety Decision
 The safety path should have higher priority than camera and low-priority traffic.

6. Isolation of Low-Priority Devices

Low-priority devices should use a separate communication path.
 For example:
 High Priority:
 Safety Sensors → Dedicated Interrupt Path

High Bandwidth:
 Cameras → PCIe → DMA → Memory

Low Priority:
 Diagnostics → Separate I/O/DMA
 This prevents diagnostic traffic from delaying safety-critical operations.

7. Application of Concepts

PCIe

Used for high-bandwidth camera devices.
DMA
Used to move camera data into memory without requiring the CPU to copy every data block.
Priority Interrupts
Used for safety-critical sensors.
Bus/Traffic Prioritization
Safety traffic receives priority over non-critical traffic.

8. Justification

The architecture separates different traffic classes:

- Camera data → high bandwidth
- Safety data → low latency and deterministic
- Diagnostic data → low priority

This prevents a large camera data stream from blocking safety-critical events.
Conclusion
The proposed autonomous-vehicle I/O architecture uses PCIe and DMA for high-bandwidth camera data, while safety-critical sensors use priority-based interrupt handling. Low-priority devices are isolated so that they cannot interfere with safety communication. This provides both high throughput and deterministic safety response.

Q3. Design an I/O mechanism for an industrial monitoring system

Given Constraints

- 100 sensors continuously generate measurement data.
- The system must support real-time fault detection.
- Interrupt coalescing must reduce processor overhead.
- Critical fault signals must receive immediate interrupt service.

1. Problem Understanding

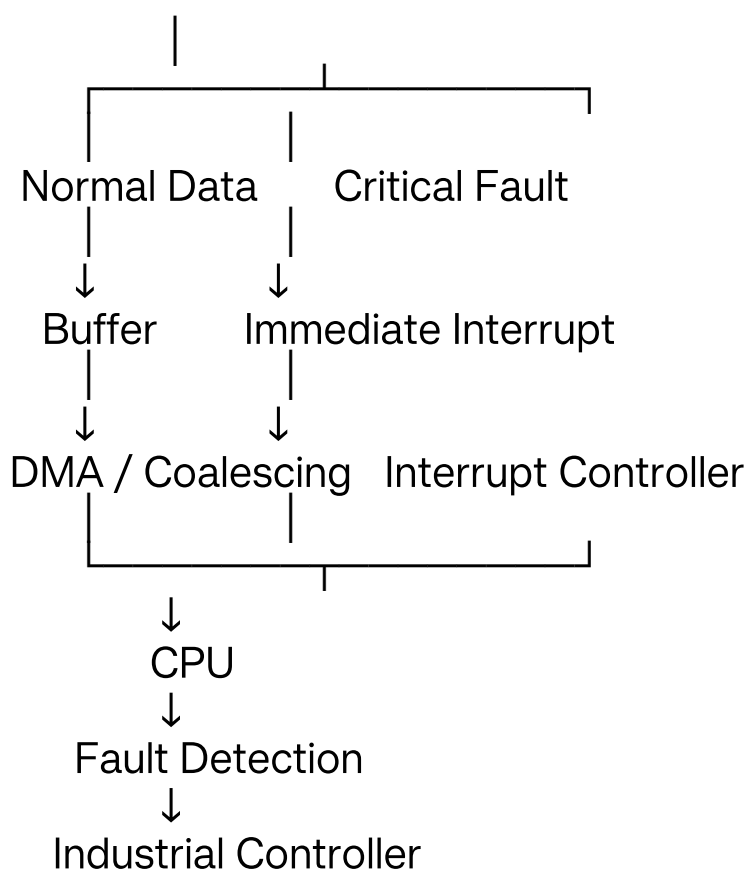
The industrial monitoring system continuously receives data from 100 sensors.
If every sensor generates an interrupt for every measurement, the CPU may spend excessive time servicing interrupts.
Therefore, the architecture needs two different mechanisms:

- Interrupt coalescing for normal sensor data.
- Immediate interrupts for critical faults.

2. Constraint Analysis

Requirement	Priority	Technique
100 continuous sensors	High	DMA/buffering
Normal measurements	Medium	Interrupt coalescing
Real-time fault detection	Critical	Fast processing
Critical fault signal	Highest	Immediate interrupt
CPU overhead	Must be reduced	Coalescing + DMA

3. Proposed Architecture
100 Sensors



4. Interrupt Coalescing

Instead of generating an interrupt for every sensor measurement, the system waits until a configurable number of measurements have accumulated.

For example:

Sensor data



Data 1

Data 2

Data 3

Data 4

Data 5



One interrupt

Thus, 5 data events can be handled using one interrupt instead of five.

Example

Suppose 100 sensor events would otherwise produce:

If interrupt coalescing processes 10 measurements per interrupt:

So the number of interrupts is reduced from:

This reduces CPU interrupt-processing overhead.

5. Critical Fault Handling

Critical faults cannot wait for a batch.

Therefore:

Critical Fault



Immediate Interrupt



High-Priority ISR



Fault Analysis



Emergency Action

This ensures real-time fault response.

6. Application of Concepts

Interrupt Coalescing
Used for normal sensor measurements.
Immediate Interrupt
Used for critical fault signals.
DMA
Can transfer batches of sensor data to memory while reducing CPU involvement.
Priority Interrupts
Critical faults receive the highest priority.

7. Justification

Using an interrupt for every measurement would create significant CPU overhead with 100 continuously operating sensors.
Interrupt coalescing reduces unnecessary interrupt processing, while the separate emergency interrupt path ensures that critical faults are never delayed by normal data batching.
Conclusion
The proposed system uses interrupt coalescing for normal sensor data and immediate high-priority interrupts for critical faults. This provides an effective balance between low CPU overhead and real-time fault detection.

Q4. Construct a high-speed network interface subsystem

- Given Constraints
- Network traffic can reach 10 Gb/s.
 - CPU must be protected from excessive packet-processing overhead.
 - DMA must transfer packets between the network interface and memory.
 - Interrupts must be generated only after processing a configurable batch of packets.

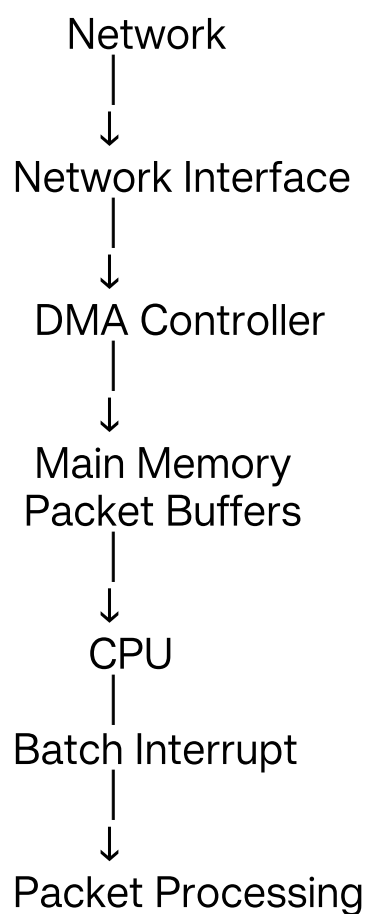
1. Problem Understanding

The network interface must process traffic up to:
Processing every packet using the CPU would create excessive overhead.
Therefore, the system should use DMA and interrupt batching.

2. Constraint Analysis

Constraint	Solution
10 Gb/s traffic	High-speed network interface
CPU overhead	DMA
Packet transfer	DMA → Memory
Interrupt frequency	Batch interrupts
Performance	Packet buffers + DMA

3. Proposed Architecture



4. Traffic Rate

Given:

Convert to GB/s:

Therefore:

The network interface must support approximately 1.25 GB of data transfer per second.

5. DMA-Based Packet Transfer

Without DMA:

Network → CPU → Memory

The CPU would be heavily involved in packet movement.

With DMA:

Network → DMA → Memory

↓

CPU

The DMA controller transfers packets directly to memory.

This frees the CPU for higher-level packet processing.

6. Interrupt Batching

Suppose the system generates one interrupt after every 32 packets.

Instead of:

Packet 1 → Interrupt

Packet 2 → Interrupt

Packet 3 → Interrupt

...

it operates as:

Packet 1

Packet 2

...

Packet 32

↓

One Interrupt

This reduces the number of interrupts by approximately a factor of 32 compared with interrupting on every packet.

7. Configurable Batch Size

The batch size can be adjusted according to traffic conditions.

Small batch

Advantages:

- Lower interrupt latency
- Faster packet processing

Disadvantage:

- More interrupts
- Higher CPU overhead

Large batch

Advantages:

- Fewer interrupts
- Lower CPU overhead

Disadvantage:

- Higher packet-processing latency

Therefore, the batch size should be selected according to the application's latency and throughput requirements.

8. Justification

DMA removes the repetitive data-copying responsibility from the CPU, while interrupt batching reduces the number of CPU interrupts.

Thus:

provides efficient high-speed packet processing.

Conclusion

The recommended subsystem uses a 10 Gb/s network interface, DMA-based packet transfer, memory buffers, and configurable interrupt batching. This reduces CPU overhead while maintaining high network throughput. The batch size can be adjusted to balance packet latency and CPU efficiency.

Q5. Design an I/O architecture for a smart healthcare monitoring system

Given Constraints

- Patient-monitoring devices operate at different sampling rates.
- Emergency signals must have the highest priority.
- Nested interrupt handling must be used for urgent events.
- Continuous monitoring data must be stored using DMA without interrupting critical processing.

1. Problem Understanding

A healthcare monitoring system receives data from multiple medical devices such as:

- Heart-rate sensors
- Blood-pressure monitors
- Oxygen-level sensors
- Temperature sensors

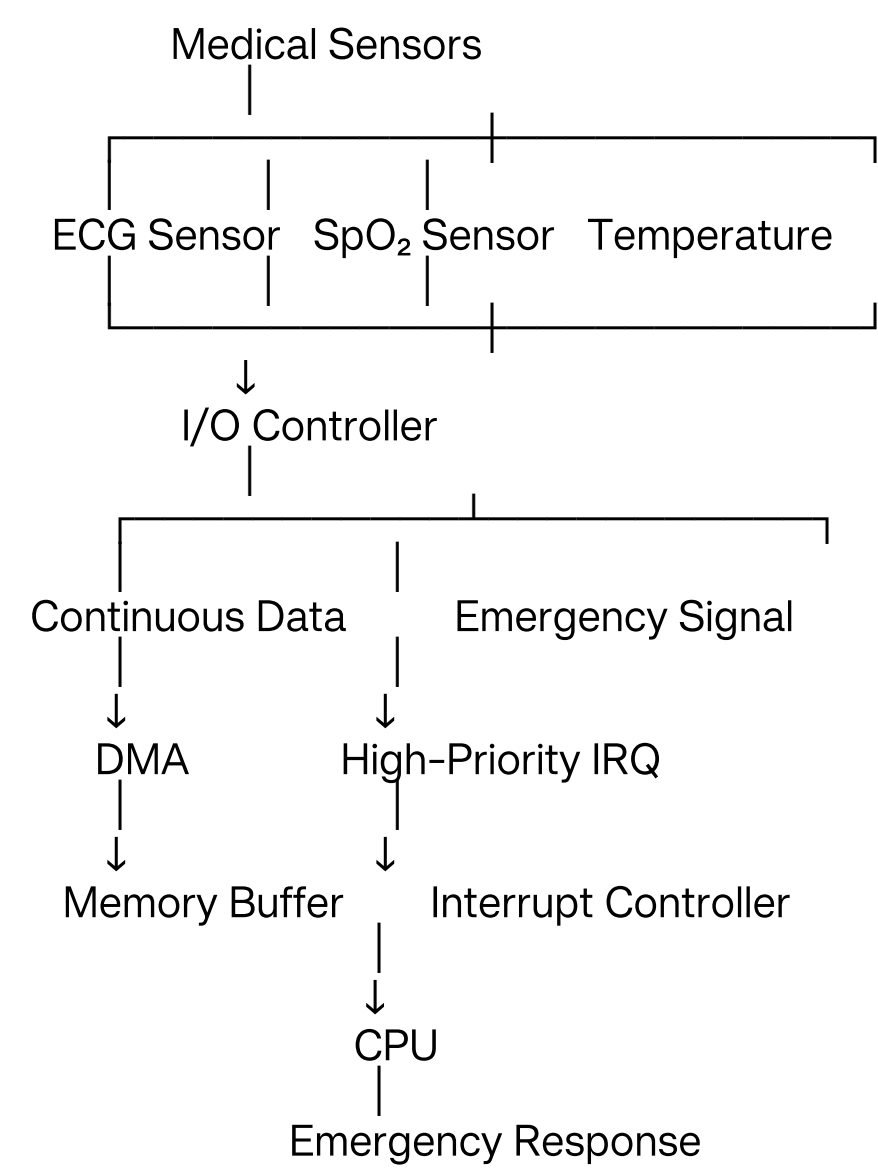
Different devices produce data at different sampling rates.

Emergency events must be processed immediately, while continuous monitoring data should be transferred efficiently without placing excessive load on the CPU.

2. Constraint Analysis

Requirement	Priorit y	Technique
Different sampling rates	High	Buffers + DMA
Emergency signals	Highe st	High-priority interrupt
Urgent events	Critica l	Nested interrupts
Continuous data	Norma l	DMA
CPU overhead	Low	DMA
Real-time response	Critica l	Priority interrupt

3. Proposed Architecture



4. Different Sampling Rates

Suppose:

- ECG → high sampling rate
- Temperature → lower sampling rate
- Blood pressure → periodic sampling

Each device can have a separate buffer.

ECG Buffer

SpO₂ Buffer

Temperature Buffer

BP Buffer

DMA can transfer the accumulated data to memory.

5. Emergency Signal Handling

Emergency signals should bypass normal data processing.

For example:

Critical abnormal reading



High-priority interrupt



Interrupt Controller



CPU



Emergency Processing



Alert / Medical Action

6. Nested Interrupt Handling

Nested interrupts allow a higher-priority interrupt to interrupt a lower-priority interrupt service routine.

Example:

Normal Sensor ISR



Emergency Interrupt



Emergency ISR



Emergency Processing Complete



Return to Normal ISR

This is important in healthcare systems because an emergency condition cannot wait for normal monitoring data processing to finish.

7. DMA for Continuous Monitoring

Normal monitoring data is transferred using DMA:

Sensor



I/O Controller



DMA



Memory Buffer



Storage / Monitoring Software

The CPU does not need to process every data transfer.

This leaves CPU resources available for emergency processing.

8. Priority Structure

A suitable priority scheme is:

Priority 1 → Emergency / Life-threatening event

Priority 2 → Critical patient abnormality

Priority 3 → Normal monitoring

Priority 4 → Logging / Diagnostic data

The highest-priority interrupt can preempt a lower-priority operation.

9. Justification

The architecture separates critical events from continuous monitoring traffic.

Emergency data receives immediate CPU attention through nested high-priority interrupts, while normal sensor data is transferred using DMA.

This prevents continuous monitoring traffic from delaying emergency response.

Conclusion

The proposed healthcare I/O architecture uses DMA for continuous monitoring data and nested priority interrupts for emergency events. Different sensor sampling rates are handled using separate buffers, while emergency signals receive immediate CPU attention. This architecture provides low CPU overhead, fast emergency response, and reliable continuous monitoring.